

Kiterjesztett valóság alapú objektumfelismerő rendszer

Önálló Laboratórium

Dátum: 2026. 05. 31.

Név / Neptun: Gál Tamás / Y5AZFI

Konzulens: Dr. Ekler Péter

Téma: Android alapú szoftverfejlesztés

1. Bevezetés, témaválasztás aktualitása

1.1 Technológiai kontextus és fejlődés

Az elmúlt évtizedben a mobil számítástechnika paradigmaváltáson ment keresztül. Az okostelefonok hardveres teljesítményének növekedése lehetővé tette, hogy korábban csak nagy teljesítményű szervereken futtatható algoritmusok kerüljenek a felhasználók zsebébe. Ezzel párhuzamosan a kiterjesztett valóság (Augmented Reality – AR) technológiája túllépett a szórakoztatóipari alkalmazásokon.

A mai modern mobileszközök már nem csupán ablakot nyitnak a digitális tartalom felé, hanem képesek a fizikai világ mélységi és strukturális értelmezésére. A Google ARCore és a mesterséges intelligencia alapú képfeldolgozó könyvtárak (mint az ML Kit) (szinergiája) együttműködése olyan új típusú alkalmazások fejlesztését teszi lehetővé, amelyek a digitális információkat közvetlenül valós tárgyakhoz képesek rendelni.

1.2 A téma aktualitása és ipari jelentősége

A kiterjesztett valóság alapú objektumfelismerés napjainkban számos stratégiai ágazatban bír kiemelt jelentőséggel:

- **Ipar 4.0 és karbantartás:** A gyárakban vagy háztartási környezetben az összetett gépi berendezések, és azok alkatrészeinek automatikus felismerése és a hozzájuk tartozó szervizinformációk térbeli megjelenítése drasztikusan csökkentheti a hibajavítási időt és a hibaarányt.
- **Logisztika és raktározás:** Az áruk intelligens felismerése és a raktári útvonalak vizuális kijelzése a fizikai térben hatékonyabbá teheti a csomagkezelést és a készletgazdálkodást.

- **Asszisztív technológiák:** A látássérültek számára készült segédeszközök terén a környezet felismerése és a tárgyak távolságának, illetve típusának hangalapú vagy haptikus visszajelzése alapvetően javíthatja az életminőséget.

A piaci igény tehát egyre erősebb olyan rendszerek iránt, amelyek nemcsak passzív megfigyelői a környezetnek, hanem aktívan, térbeli kontextusban képesek értelmezni azt.

1.3 Célkitűzések

A projekt elsődleges célja egy olyan Android alapú prototípus alkalmazás fejlesztése, amely demonstrálja a valós idejű objektumdetekció és a kiterjesztett valóság integrációjának (előnyeit). A fejlesztés során kiemelt figyelmet fordítunk az alábbiakra:

- **Valós idejű feldolgozás:** Olyan architektúra kialakítása, amely képes a nagy felbontású kamerafolyam elemzésére anélkül, hogy a felhasználói élményt (a vizuális folyamatosságot) rontaná.
- **Térbeli perzisztencia (Spatial Persistence):** A felismert objektumok koordinátáinak leképezése a 3D-s világtérbe, ezzel biztosítva, hogy a virtuális címkék és jelölések akkor is a fizikai tárgyhoz rögzítve maradjanak, ha a felhasználó elmozdul vagy elfordítja a készüléket.
- **Robosztus környezetérzékelés:** A környezet pontfelhő alapú (Point Cloud) reprezentációjának felhasználása a pontosabb pozicionálás érdekében.

Jelen beszámoló a fejlesztés ezen fázisában elért eredményeket, a választott technológiákat és a megvalósított rendszer felépítését mutatja be.

2. Feladat/problémakör bemutatása

2.1 A hagyományos 2D detekció korlátai

A legtöbb mobil alapú képfelismerő rendszer a kamera képén (2D síkban) határolókeretekkel (bounding boxes) jelöli meg a talált tárgyakat. Ez a megközelítés azonban nem elegendő egy kiterjesztett valóság alapú rendszernél, mivel:

- **Hiányzik a mélységérzékelés:** A rendszer nem tudja, milyen messze van a tárgy, így nem tudja valós méretben vagy takarásban (occlusion) megjeleníteni az információkat.
- **Nincs térbeli perzisztencia:** Ha a felhasználó elfordítja a kamerát, a 2D-s koordináták érvényüket veszítik. A cél az, hogy a felismert tárgyhoz rendelt címke akkor is a "helyén maradjon", ha az adott tárgy éppen nincs a kamera látóterében.

2.2 A kitűzött részfeladatok

A probléma megoldása érdekében a fejlesztést három fő technikai pillérre bontottuk:

2.2.1 A környezet 3D-s reprezentációja

Ahhoz, hogy a virtuális elemeket elhelyezhessük, a szoftvernek értenie kell a tér geometriáját. Erre a **Point Cloud (pontfelhő)** technológiát alkalmazzuk. Az ARCore segítségével a rendszer vizuális jellegzetességek alapján pontokat azonosít a térben, és meghatározza azok X, Y, Z koordinátáit. Ez a pontfelhő szolgál a környezet digitális vázaként, amelyre a későbbiekben építkezhetünk.

2.2.2 Nagy teljesítményű képfeldolgozó algoritmus integrálása

Az objektumfelismerés számításigényes folyamat. A feladat egy olyan aszinkron pipeline kialakítása, amely képes:

- A kamera képkockáinak valós idejű elemzésére.
- A processzor és a grafikus gyorsító (GPU) közötti terheléelosztásra.
- A detekciós eredmények (címke, bizalmi szint) gyors továbbítására a megjelenítő réteg felé anélkül, hogy a képfrissítés (rendering) akadozna.

2.2.3 Leképezés a 3D-s világtérbe (Anchoring)

A legösszetettebb feladat a 2D-s detekciós pont (ahol az algoritmus a tárgyat látja a beérkező képen) leképezése a 3D-s világtérbe. Ez a folyamat a következő lépésekből áll:

- **Raycasting:** Virtuális sugár indítása a kamera optikai középpontjából a detektált tárgy irányába.
- **Metszéspon keresése:** A sugár és a korábban említett pontfelhő által detektált síkok metszésponkjának meghatározása.
- **Anchor elhelyezése:** Egy fix referenciapont létrehozása az ARCore világtérben, amelyhez a 3D-s vizuális elemek (nyilak, szövegbuborékok, és egyéb objektumok) rögzíthetők.

Ezen részfeladatok együttes megoldása teszi lehetővé, hogy a felhasználó egy stabil, a valósághoz (szervesen / szorosan) illeszkedő információs réteget kapjon a készüléke kijelzőjén.

3. Félév ütemezése

A fejlesztési folyamat egy több féléven átívelő projekt részeként valósul meg, melynek során az elméleti megalapozástól a működő prototípus elkészítésén át jutunk el a teljes körű végleges megvalósításig.

A munka első szakasza az elméleti kutatásról és a technológiák kiválasztásáról szól, majd ezt követte az objektumfelismerés és az AR technológia integrálása, valamint a rendszer architektúrájának kidolgozása.

- 1–3. hét: Előkészületek és utánajárás
 - **Irodalomkutatás:** A Google ARCore keretrendszernek, valamint az MLKit technológiának való utánajárás.
 - **Környezet kialakítása:** Az Android Studio fejlesztői környezet konfigurálása, alapvető Android-fejlesztési koncepciók (Lifecycle, Permissions, Coroutines) elsajátítása.
 - **Proof of Concept (PoC):** Egy kezdetleges alkalmazás létrehozása, amely képes képek/objektumok felismerésére (2D síkban).
- 4–6. hét: Környezetérzékelés és vizualizáció
 - A pontfelhő (Point Cloud) adatok kinyerése és megjelenítése.
 - A PointCloudRender osztály megvalósítása, amely lehetővé teszi a felhasználó számára, hogy lássa az ARCore által érzékelt mélységi adatokat, és a detektált síkok reprezentációját (kék pontok formájában).
- 7–9. hét: Objektumdetekciós pipeline
 - A Google ML Kit integrálása a projektbe.
 - Az aszinkron képfeldolgozási logika kidolgozása: a kamera frame-ek továbbítása a gépi tanulási modellnek anélkül, hogy a renderelő szál megakadna.
- 10–12. hét: Térbeli illesztés és interakció
 - A detektált 2D-s koordináták 3D-s térbe való leképezése (Raycasting).
 - Az Anchor-menedzsment kidolgozása: virtuális címkék és háromdimenziós modellek (például nyilak) rögzítése a felismert tárgyakhoz.
 - A felhasználói felület (UI) finomhangolása.

- 13–14. hét: Tesztelés és dokumentáció
 - A rendszer stabilitásának és pontosságának tesztelése különböző fényviszonyok és tárgytipusok mellett. A PointCloudRender osztály felhasználása a térbeli pontosság ellenőrzéséhez.
 - A projektről készített beszámoló véglegesítése.

4. Felhasznált technológiák

A fejlesztés során a modern Android-ökoszisztéma legfrissebb megoldásait alkalmaztam, különös tekintettel a valós idejű adatfeldolgozásra.

4.1 Programozási nyelv és környezet

A projekt Kotlin nyelven készült, amely az Android hivatalos és elsődleges nyelve. A Kotlin használata mellett szólt a tömör szintaxis, a típusbiztonság és a Coroutines (korutinok) natív támogatása. Utóbbi kritikus szerepet játszik a projektben, mivel lehetővé teszi a számításigényes gépi tanulási feladatok háttérszálon történő futtatását, megakadályozva a felhasználói felület és az AR-megjelenítés akadozását.

4.2 Google ARCore (v1.41.0)

Az alkalmazás "szeme" az ARCore keretrendszer, amely a kiterjesztett valóság funkciókat biztosítja. Két fő képességét használtam ki:

- **Mozgáskövetés (Motion Tracking):** A telefon térbeli pozíciójának meghatározása a kamera adatai alapján.
- **Környezetérzékelés (Environmental Understanding):** A pontfelhő (Point Cloud) generálása, amely a valós világ mélységi adatait szolgáltatja.
- **Hit-testing (Ütközésvizsgálat):** Ez az a technológia, ami lehetővé teszi, hogy a képernyő egy 2D-s pontjából (ahol az ML Kit megtalálta a tárgyat) „kilőjünk” egy sugarat a 3D-s térbe, és megkeressük, hol találkozik a valósággal.
- **Anchoring (Horgonyok létrehozása):** Az ARCore képes „horgonyokat” elhelyezni a virtuális térben, amelyek fixen rögzülnek a fizikai világhoz. Ez biztosítja, hogy ha elfordítod a telefont, a címke ne „ússzon el”, hanem ott maradjon a tárgy felett.
- **Koordináta-transzformáció (Coordinate Transformation):** Ez egy matematikai híd. A kamera képe (Image pixels), a kijelző (View) és a 3D-s világtér (World space) más-más koordináta-rendszert használ. Az ARCore végzi el ezek között az átváltást.

4.3 Google ML Kit (v17.0.0)

Az objektumfelismerésért az ML Kit Object Detection and Tracking könyvtára felel. A választás azért esett erre az eszközre, mert:

- **On-device futtatás:** A képek feldolgozása helyben, az eszközön történik, így nincs szükség internetkapcsolatra, és minimális a késleltetés.
- **Custom Model támogatás:** Lehetőséget biztosít egyedi TensorFlow Lite modellek alkalmazására is.

4.4 OpenGL ES és Shader programozás

A kiterjesztett valóság elemeinek és a pontfelhőnek a megjelenítése alacsony szintű grafikai hívásokkal, OpenGL ES segítségével történik. Ehhez egyedi GLSL (OpenGL Shading Language) programokat (shadereket) alkalmaztam:

- **point_cloud.vert:** A pontok pozicionálásáért felelős vertex (csúcs) shader.
- **point_cloud.frag:** A pontok színéért és átlátszóságáért felelős fragment (töredék) shader. A pontfelhő adatait közvetlenül a GPU-nak adjuk át VertexBuffer és Mesh objektumokon keresztül, ami maximális renderelési sebességet biztosít.

4.5 Segédkönyvtárak és UI keretrendszer

- **Jetpack Compose:** A modern, deklaratív UI keretrendszer a felhasználói felület (gombok, visszajelzések) kialakításához.
- **SampleRender:** Az ARCore mintaprojektjei segítségével kialakított segédosztály, amely megkönnyíti az OpenGL erőforrások (Textúrák, Bufferek, Shaderok) kezelését.
- **OBJ Parser:** A 3D-s objektumfájlok (pl. a felismert objektumot jelző nyilak) beolvasására szolgáló könyvtár.

5.A rendszer architektúrális felépítése

A kiterjesztett valóság és a gépi tanulás integrálása komplex szoftverarchitektúrát igényel, mivel két, alapvetően eltérő sebességű folyamatot kell összehangolni: a gyorsabb (60 FPS) sebességű grafikai megjelenítést és a változó idejű (jellemzően lassabb) objektumdetekciót.

5.1 Architektúra

A rendszer tervezésekor az MVVM (Model-View-ViewModel) alapelveit követtem. A szoftver egy modern, reaktív szemléletű architektúrára épül, amely szétválasztja az Android életciklus-kezelést, a felhasználói felületet és a számításigényes AR/ML folyamatokat. A feladatköröket az alábbiak szerint választottam szét:

- **Megjelenítési réteg - Compose és OpenGL hibrid:**

Kritikus tervezési pont a kamera képének és az AR elemeknek a megjelenítése.

Bár az alkalmazás felhasználói felülete Jetpack Compose alapú, a kamera képének renderelése nem natív Compose elemekkel történik, mivel az ARCore-nak közvetlen, alacsony késleltetésű hozzáférésre van szüksége a GPU-hoz.

- **AndroidView integráció:** A Compose környezetben belül egy AndroidView komponenst használunk, amely befogadja a hagyományos GLSurfaceView-t, amely az OpenGL kontextust biztosítja.
- **Háttér renderelés:** A kamera képkockái egy speciális OpenGL textúrába érkeznek. Az ARCore session minden képkockánál frissíti ezt a textúrát, amelyet egy dedikált shader (Background Renderer) rajzol ki a kijelzőre a virtuális objektumok „mögé”. Ez biztosítja, hogy a videófolyam és a 3D-s elemek tökéletes szinkronban legyenek.

- **Logikai vezérlés:**

Az alkalmazás logikai vezérlését a MainActivity, mint orkesztrátor végzi, amely összehangolja a következő kulcsfontosságú elemek működését:

- **MainViewModel:** Az alkalmazás reaktív állapottárolója. Itt tárolja a rendszer az olyan adatokat mint, hogy éppen aktív-e a szkennelés vagy engedélyezett-e a törlés gomb. A ViewModel és az AppRenderer közötti kapcsolatot callback függvények biztosítják.
- **ARCoreSessionLifecycle:** Ez a modul a rendszer „életbiztosítása”. Ez kezeli az ARCore session létrehozását, a jogosultságkéréseket és a kritikus hibák (pl. ha nincs telepítve az ARCore) lekezelését, leválasztva ezt a bonyolult logikát a MainActivity-ről.
- **DisplayRotationHelper:** Gondoskodik arról, hogy a kamera képe és a 3D-s koordináták helyesen forduljanak el a telefon fizikai elforgatásakor szinkronban tartva a fizikai kijelzőt és a szenzoradatokat.
- **TrackingStateHelper:** Folyamatosan monitorozza az ARCore állapotát. Ha a követés megszakad (pl. túl gyors mozgás vagy kevés fény miatt), ezen keresztül küldünk jelzést a UI-nak.
- **SnackbarHelper:** A felhasználói visszajelzések (hibák, útmutatások) aszinkron megjelenítéséért felelős, biztosítva, hogy a vizuális üzenetek ne blokkolják a renderelést.

A komponensek közötti kommunikáció eseményvezérelt: az ARCore frame-update eseménye indítja el a láncreakciót, amely végigfut a logikai elemeken és a renderelőn.

- **ML Engine:**

A gépi látásért felelős modul központi eleme az MLKitObjectDetector, amely az ObjectDetector absztrakt osztály leszármazottja. Ez az osztály valósítja meg a konkrét detekciós logikát:

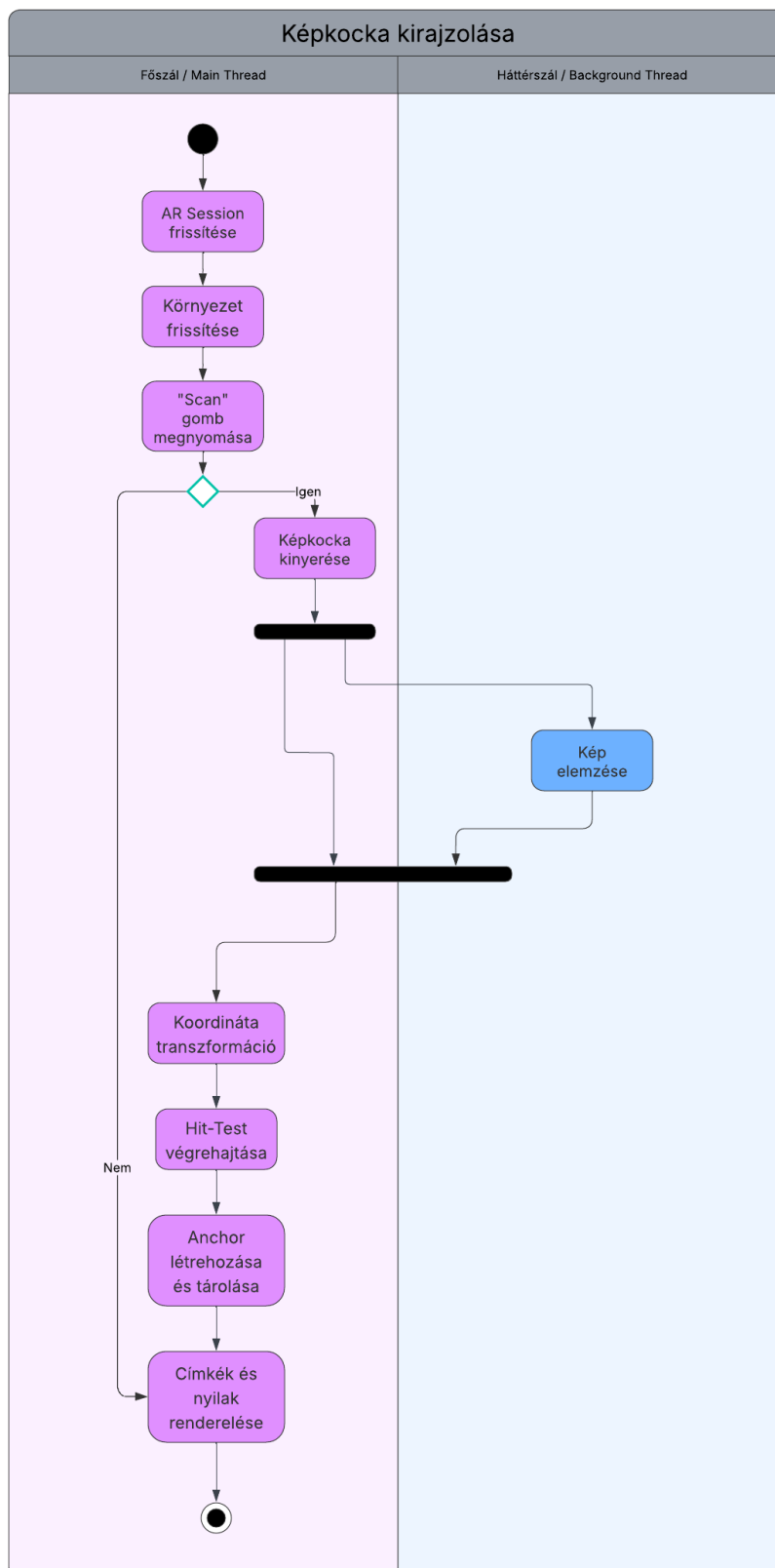
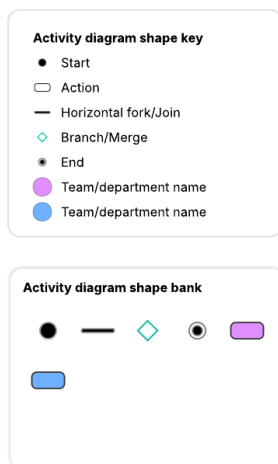
- **Képkonverzió:** Az AppRenderer-ből érkező, a kamera által biztosított YUV formátumú képet (ImageProxy) alakítja át az ML Kit számára értelmezhető InputImage formátumra.
- **Modellkonfiguráció:** Itt történik az objektumdetektáló modell konfigurálása.
- **Aszinkron feldolgozás:** Az osztály biztosítja, hogy a detekció a Dispatchers.IO szálon fusson, így a kép elemzése nem akasztja meg a kijelzést. Az eredményeket (felismert objektumok neve és pozíciója) aszinkron módon adja vissza a feldolgozónak.

- **Render Engine - AppRenderer és SampleRender:**

A vizuális kimenet generálása két szinten történik:

- **SampleRender:** Ez egy alacsony szintű absztrakciós réteg, amely megkönnyíti az OpenGL erőforrások (shaderek, pufferek, textúrák) biztonságos és hatékony kezelését, függetlenül a kódot a redundáns GL hívásoktól.
- **AppRenderer:** Ez az osztály a projekt „grafikai agya”. Itt történik a nézeti mátrixok (View/Projection Matrix) kiszámítása, ami garantálja, hogy a virtuális tárgyak a kamera mozgását követve a helyükön maradjanak. Feladata a színtér (scene) összeállítása. Valamint itt fut az onDrawFrame ciklus, amely minden egyes képkockánál:
 - Frissíti az ARCore állapotát.
 - Lekéri a kamera nézeti és projekciós mátrixait.
 - Vezérli a PointCloudRender-t a környezeti pontfelhőhöz.
 - Koordinálja a felismert tárgyak feletti 3D-s nyilak és szöveges címkék (LabelRender, ObjectRenderer) kirajzolását.

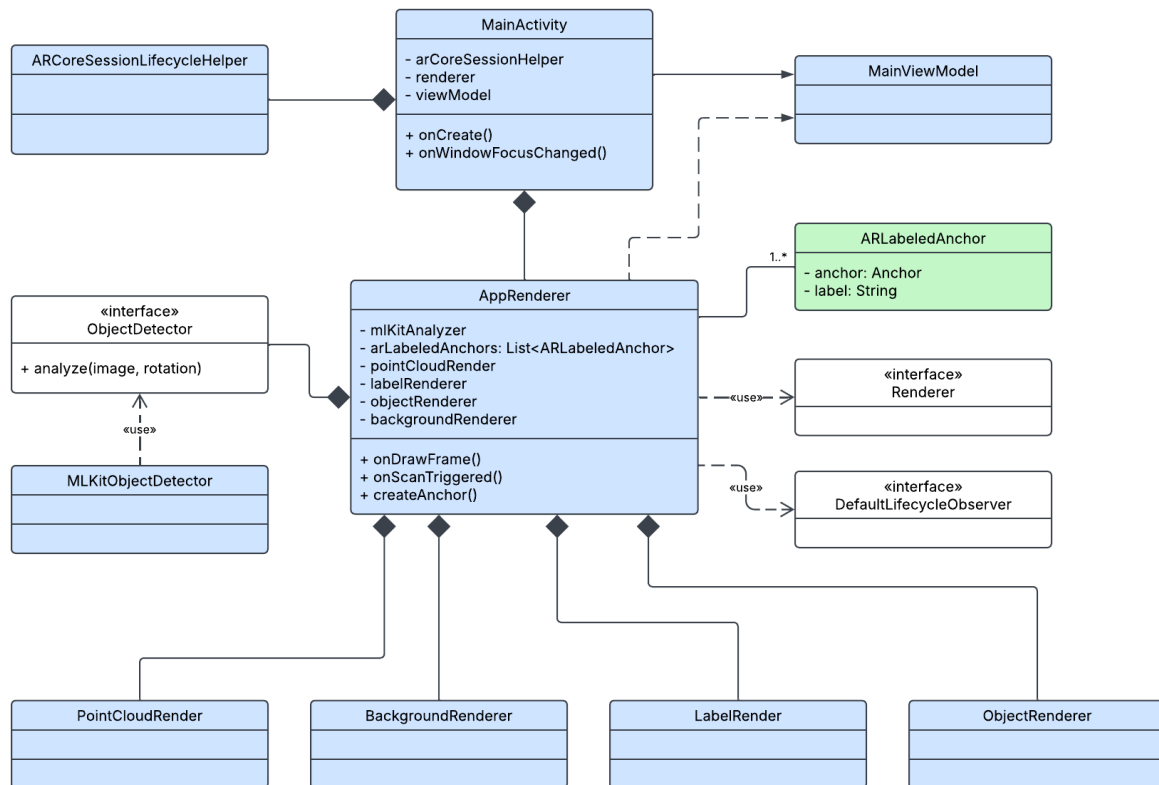
5.2 Képkocka megjelenítése



A diagram logikai menete:

1. Indítás (Főszál): onDrawFrame esemény érkezik.
2. AR Session frissítése: Az ARCore Session frissítése (session.update()), az aktuális Frame objektum lekérése.
3. Környezet frissítése: A háttér (backgroundRenderer) és a pontfelhő (PointCloudRender) azonnali kirajzolása. Ez biztosítja a folytonosságot.
4. Megnyomta a felhasználó a "Scan" gombot? (scanButtonWasPressed)
 - I. NEM: Ugrás a 9. ponthoz (meglévő horgonyok rajzolása).
 - II. IGEN: Képkocka kinyerése (frame.tryAcquireCameraImage()).
5. Kamera kép kinyerése: (tryAcquireCameraImage()): Ha igen, kivesszük az aktuális képet a bufferből.
6. Elágazás:
 - I. Főszál folytatja: A renderelés nem áll meg, a felhasználó továbbra is látja a mozgó kamera képét.
 - II. Háttérszál indul (Coroutine - Dispatchers.IO): Az MLKitObjectDetector.analyze() meghívása a kinyert képpel.
7. Kép elemzése: Az ML Kit végrehajtja az objektumdetekciót és osztályozást.
Eredmény: DetectedObjectResult lista.
8. Eredmény visszaadása (Join): A háttérszál átadja az eredményeket az AppRenderer-nek (objectResults).
9. Térbeli illesztés:
 - I. Koordináta transzformáció: Kép-pixel koordináták leképezése kijelző-koordinátákká.
 - II. Hit-Test végrehajtása: Virtuális sugár indítása a kijelző adott pontjáról. Ha a sugár találkozik egy valós felülettel vagy pontfelhővel -> Anchor (horony) létrehozása.
 - III. Anchor létrehozása és tárolása: Az Anchor és a Label elmentése az arLabeledAnchors listába.
10. Címkék és nyilak renderelése: Iterálás az összes mentett arLabeledAnchors elemen.
 - I. labelRenderer.draw(): Felirat kirajzolása.
 - II. objectRenderer.draw(): 3D-s nyíl kirajzolása a horony pozíciójában.
11. Ciklus vége: A folyamat előlről kezdődik a következő képkockával.

5.3 Fontosabb osztályok kapcsolata



6. Megvalósítás részletei és tapasztalatok

A megvalósítás során a legnagyobb kihívást a precizitás és a sebesség közötti egyensúly megteremtése jelentette. Az alábbiakban a három legfontosabb technikai modul részleteit ismertetem.

6.1 Aszinkron képfeldolgozási pipeline

A felhasználói élmény folytonossága érdekében a gépi tanulási modellt nem a főszálon futtatjuk. A AppRenderer.kt fájlban a Kotlin Coroutines segítségével valósítottuk meg az aszinkron hívást:

```

if (scanButtonWasPressed) {
    scanButtonWasPressed = false
    val cameraImage = frame.tryAcquireCameraImage()
    if (cameraImage != null) {
        launch(context = Dispatchers.IO) {
            val cameraId = session.cameraConfig.cameraId
            val imageRotation = displayRotationHelper.getCameraSensorToDisplayRotation(cameraId)
            objectResults = currentAnalyzer.analyze(cameraImage, imageRotation)
            cameraImage.close()
        }
    }
}
}

```

Tapasztalat: A Dispatchers.IO használatával a detekció alatt is megmarad az elvárt képfrissítés. A cameraImage.close() hívás kritikus, mert ha elmarad, az ARCore nem tud újabb képkockát biztosítani, és a rendszer lefagy.

6.2 2D-3D koordináta-transzformáció és Raycasting

Az ML Kit a tárgyakat pixel-koordinátákban adja meg (2D), de nekünk a 3D térben kell elhelyeznünk a jelölőt. Ehhez egy kétlépcsős folyamatot alkalmazunk:

- **Leképezés a nézetre:** A kamera képének pixeleit átalakítjuk a kijelző (View) koordinátáira a frame.transformCoordinates2d függvénnyel.
- **Raycasting (Hit-test):** Virtuális sugarat indítunk a pontfelhő és a detektált síkok irányába.

```

fun createAnchor(xImage: Float, yImage: Float, frame: Frame): Anchor? {
    convertFloats[0] = xImage
    convertFloats[1] = yImage
    frame.transformCoordinates2d(
        inputCoordinates = Coordinates2d.IMAGE_PIXELS,
        inputVertices2d = convertFloats,
        outputCoordinates = Coordinates2d.VIEW,
        outputVertices2d = convertFloatsOut
    )

    val hits = frame.hitTest(xPx = convertFloatsOut[0], yPx = convertFloatsOut[1])
    val result = hits.getOrNull(index = 0) ?: return null
    return result.trackable.createAnchor(result.hitPose)
}

```

Tapasztalat: A Hit-test eredményessége nagyban függ a pontfelhő sűrűségétől. Ha a felhasználó túl gyorsan mozgatja a telefont, a pontfelhő még ritka, és a horgony (Anchor) nem hozható létre.

6.3 Dinamikus pontfelhő renderelés

A PointCloudRender osztály felelős a pontok hatékony kezeléséért. A megvalósítás során a legnagyobb kihívást a CPU-GPU közötti adatátvitel optimalizálása jelentette. A kód az alábbi optimalizált frissítési logikát használja:

```
fun drawPointCloud(
    render: SampleRender,
    pointCloud: PointCloud,
    modelViewProjectionMatrix: FloatArray
) {
    if (pointCloud.timestamp > lastPointCloudTimestamp) {
        pointCloudVertexBuffer.set(pointCloud.points)
        lastPointCloudTimestamp = pointCloud.timestamp
    }
    pointCloudShader.setMat4( name = "u_ModelViewProjection", values = modelViewProjectionMatrix)
    render.draw(pointCloudMesh, pointCloudShader)
}
```

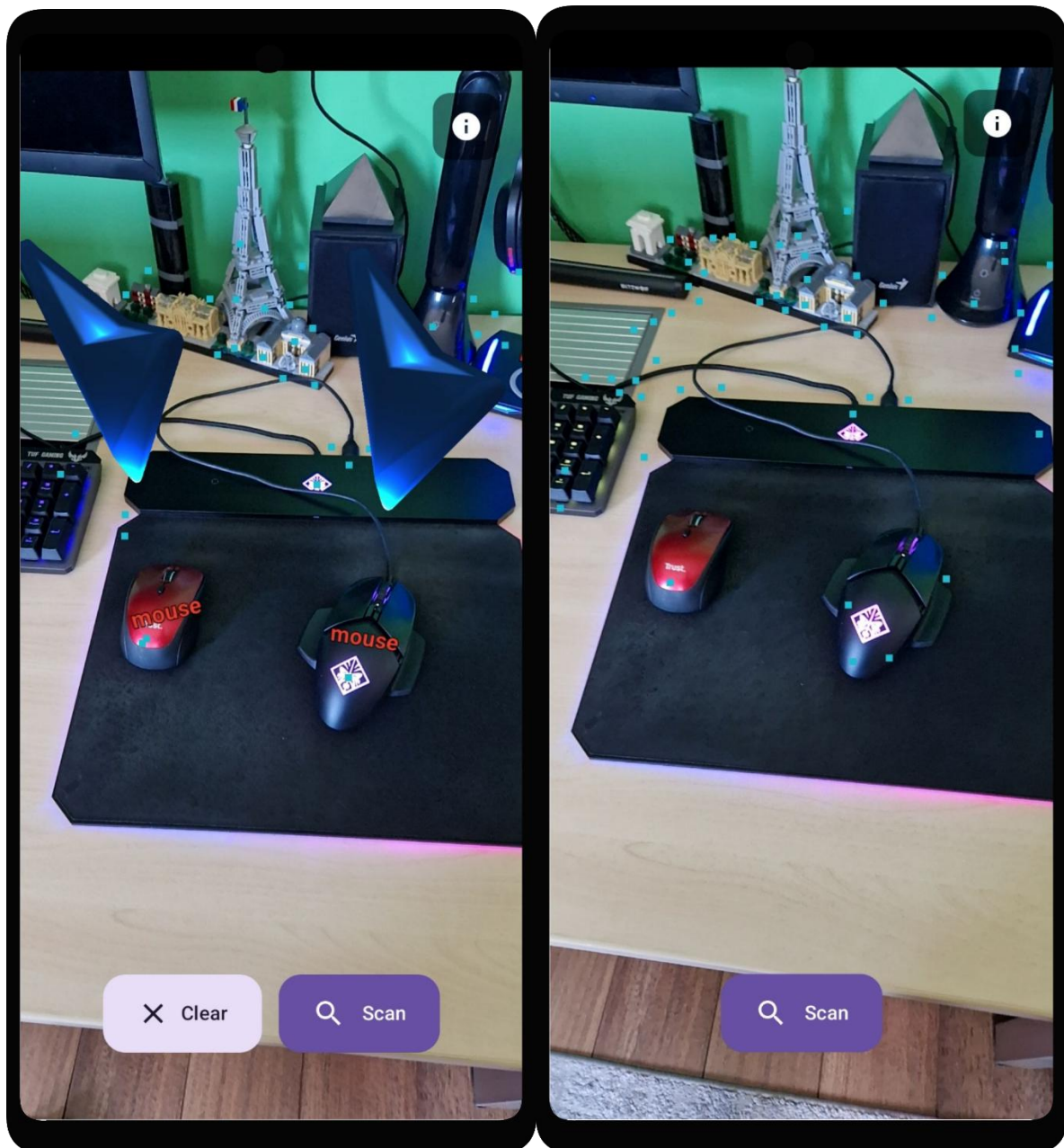
Fontos technikai részlet a VBO Optimalizációja: A pointCloud.timestamp ellenőrzése kritikus fontosságú. Mivel a GPU memória írása erőforrásigényes, ezzel a feltétellel elkerüljük a felesleges adatátvitelt azokban a képkockákban, ahol az ARCore nem szolgáltatott új mélységi adatot.

6.4 Tapasztalatok és mérések

A tesztelést egy Samsung Galaxy S22 eszközön végeztem.

- **Detekciós késleltetés:** Az ML Kit átlagosan 45-70 ms alatt ismeri fel a tárgyakat. Ez elegendően gyors ahhoz, hogy ne tűnjön szaggatottnak a folyamat.
- **Pontosság:** A felismerés 0,5 - 1,5 méteren belül a legstabilabb. Távolabbi tárgyak esetén a Hit-test gyakran sikertelen a felbontás korlátai miatt.
- **Korlátok:** A felismerés vízszintes felületek esetén a leghatékonyabb. Függőleges vagy textúrázatlan felületek esetén a síkok feltérképezése korlátozott, vagy hibás lehet, ami pontatlan címke és objektum elhelyezést eredményezhet.

7. Képernyőképek az elkészült megoldásról



8. Továbbfejlesztési lehetőségek

A félév során sikerült egy stabil, valós idejű objektumdetekcióra és térbeli horgonyzásra képes prototípust létrehozni. Ugyanakkor az AR és az ML területe számos olyan fejlett megoldást kínál, amelyek integrálása a következő félév(ek) feladata lehet.

8.1 *Specifikus tárgyfelismerés egyedi modellekkel (TFLite)*

Jelenleg a rendszer a Google egy általános objektummodelljét használja, amely korlátozott számú kategória (pl. bútor, ruházat, használati tárgy) felismerésére képes.

- **Cél:** Saját TensorFlow Lite modellek betöltése, amelyek egy konkrét szakterület objektumaira vannak optimalizálva (például egy laboratórium specifikus mérőműszerei vagy egy gépterem alkatrészei).
- **Várható eredmény:** Pontosabb osztályozás és akár ipari környezetben való alkalmazhatóság.

8.2 Mélységi okklúzió (*Depth API*)

Jelenleg a virtuális nyilak és feliratok mindig a kamera képe "előtt" látszanak, még akkor is, ha egy valós tárgy (például egy fal vagy egy másik bútor) fizikailag takarná őket.

- **Cél:** Az ARCore Depth API mélyebb integrációja a renderelési folyamatba.
- **Várható eredmény:** Realisztikusabb megjelenítés, ahol a digitális objektumok valóban "eltűnnek" a fizikai akadályok mögött, drasztikusan növelve a jelenlétérzetet (immersion).

8.3 Adatbázis alapú adatmegjelenítés

Az alkalmazás kibővítése egy adatbázissal, valamint hozzá tartozó webes felülettel. Lehetővé téve, hogy ezen keresztül a felhasználók az integrált modell által felismerni képes objektumokhoz információkat rendelhessenek, melyek az alkalmazásban az adott objektum detektálását követően hozzáférhetővé válnának. Például egy háztartási eszköz felismerése esetén hozzáférhető lehet annak műszaki leírása.

A projekt során lefektetett moduláris lehetővé teszik ezen funkciók inkrementális fejlesztését. A következő félév fókuszában várhatóan az adatbázis alapú információ megjelenítés, valamint az egyedi modellek integrálása fog állni, közelebb hozva ezzel a prototípust egy valós, ipari körülmények között is használható termékhez.